Flake Exchange Router

Security Audit Report

March 17, 2026

BugBlow

Contents

1. INTRODUCTION

1.1 Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, investment advice, endorsement of the platform or its products, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only. The information presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of the Client. If you are not the intended recipient(s) of this document, please note that any disclosure, copying or dissemination of its content is strictly forbidden.

1.2 Security Assessment Methodology

BugBlow utilized a widely adopted approach to performing a security audit. Below is a breakout of how our team was able to identify and exploit vulnerabilities.

1.2.1 Code and architecture review:

- Review the source code.
- Read the project's documentation.
- Review the project's architecture.

Stage goals

- Build a good understanding of how the application works.

1.2.2 Check code against vulnerability checklist and tools:

- Understand the project's critical assets, resources, and security requirements.
- Manual check for vulnerabilities based on the auditor's checklist.
- Run the automated Savant Chat AI tool
- Run static analyzers.

Stage goals

- Identify and eliminate typical vulnerabilities (gas limit, replay, flash-loans attack, etc.)

1.2.3 Threat & Attack Simulation:

- Analyze the project's critical assets, resources and security requirements.
- Exploit the found weaknesses in a safe local environment.
- Document the performed work.

Stage goals

- Identify vulnerabilities that are not listed in static analyzers that would likely be exploited by hackers.
- Develop Proof of Concept (PoC).

1.2.4 Report found vulnerabilities:

- Discuss the found issues with developers
- Show remediations.
- Write an initial audit report.

Stage goals

- Verify that all found issues are relevant.

1.2.5 Fix bugs & re-audit code:

- Help developers fix bugs.
- Re-audit the updated code again.

Stage goals

- Double-check that the found vulnerabilities or bugs were fixed and were fixed correctly.

1.2.6 Deliever final report:

- The Client deploys the re-audited version of the code
- The final report is issued for the Client.

Stage goals

- Provide the Client with the final report that serves as security proof.

Vulnerability Risk Classification

All vulnerabilities discovered during the audit are classified based on a combination of their potential **impact** and **likelihood** of exploitation. The resulting severity is determined using the following risk matrix:

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** — Direct theft or permanent loss exceeding 1% of the protocol's TVL, or permanent blocking of user funds (>1%) without the possibility of withdrawal or recovery.
- **Medium** — One-time theft of up to 1% of the protocol's TVL, or fund locking that can only be resolved through a contract upgrade, or indirect losses (e.g., lost yield, blocked rewards) exceeding 1% where eventual recovery is possible.
- **Low** — Contract malfunction or temporary lock that can be resolved by the administrator without a contract upgrade, with financial impact below 0.1% of TVL or negligible effect on end users.

Likelihood

- **High** — Exploitation requires no privileged access and no rare preconditions. Any external actor can trigger the vulnerability under common contract states. Estimated probability of occurrence: >50% within a year.
- **Medium** — Exploitation requires specific but realistic conditions, such as a particular contract state, market movement, or trusted actor involvement. Estimated probability: 10–30% within a year.
- **Low** — Exploitation requires an extremely rare combination of conditions, a compromised privileged actor, or conditions with <5% probability of occurring within a year.

Action Required

- **Critical** — Must be fixed immediately before deployment or as an emergency upgrade.
- **High** — It is strongly recommended to fix in the nearest release cycle.
- **Medium** — Recommended to fix to improve protocol security and stability.
- **Low** — Advisable to fix for overall robustness; low urgency.

Finding Status

- **Open** — The issue has not been fixed yet.
- **Fixed** — The issue has been fixed.
- **Acknowledged** — The team has reviewed the finding but has not yet implemented changes, or the finding does not require code modifications.

1.3 Project Overview

Project description is

1.4 Project Dashboard

Project Summary

Title	Flake Exchange Router
Client name	Flake Exchange
Project name	Flake Exchange Router
Timeline	25.02.2026 - 02.03.2026
Re-audit	16.03.2026
Number of Auditors	1

Project Audited Log

Date	Project Link
20.02.2026	`https://etherscan.io/address/0x532f3e526228ff5bc5b80faad119380829326c08`
16.03.2026	`https://etherscan.io/address/0xbb680495dab6dc61598204db393ae66246a73403`

Project Scope

The audit covered the following smart contract files.

File name
Router.sol
RouterV2.sol
proxy/FlakeRouter.sol
Re-audit scope (updated contracts)
Router.sol
FlakeRouter.sol

1.5 Summary of findings

Figure 1. Findings chart

Severity	# of Findings
Critical	3
High	0
Medium	3

Severity	# of Findings
Low	0

2. FINDINGS REPORT

2.1 Critical

C-01: Unlimited Allowance for All Users Funds

C-01	Unlimited Allowance for All Users Funds
Severity	Critical
Status	Acknowledged
Location	`contracts/Router.sol: L198-206`

Description

Function `_executeSwapWithPermitInternal` requests `type(uint256).max` as the allowed amount instead of the actual `_amountIn` when calling `IERC20Permit.permit()`. After the swap, this unlimited allowance remains forever — the contract does not revoke it.

This means the Router receives the right to move any amount of that token from the user's wallet indefinitely.

Even without C-02, this issue becomes more severe due to the risk of a malicious proxy upgrade (`upgradeTo`) if the contract's administrative key is compromised.

Re-audit Note (March 16, 2026)

The updated contract attempts to fix this by first calling `permit` with `_amountIn`. However, if that call fails, it falls back to `type(uint256).max`:

```
try IERC20Permit(_tokenIn).permit(
    msg.sender, address(this), _amountIn, ...
) {
    permitSucceeded = true;
} catch {}

// Fallback: still requests unlimited allowance
if (!permitSucceeded) {
    try IERC20Permit(_tokenIn).permit(
        msg.sender, address(this), type(uint256).max, ...
    ) {
        permitSucceeded = true;
    } catch { revert("Permit Failed"); }
}
```

This fallback completely negates the fix. If the user's signature was created for `type(uint256).max` (e.g. by a frontend that still uses max approval), the first `try` will fail (signature mismatch) and the second `try` will succeed — granting unlimited allowance exactly as before. The fallback branch must be removed entirely so that only exact `_amountIn` permits are accepted.

Suggested Fix

Request permit for only the necessary `_amountIn`:

```
try IERC20Permit(_tokenIn).permit(
    msg.sender,
    address(this),
    _amountIn,
    _deadline,
    _v,
    _r,
    _s
) {
```

Proof of Concept

```
// User swaps 100 USDC to ETH through permit
router.executeSwapWithPermit(
    target, data, USDC, address(0), 100e6, deadline, v, r, s
);

// After the swap:
USDC.allowance(user, address(router));
// Returns: 115792089237316195423570985008687907 8...
// (type(uint256).max - this is forever)

// Any future malicious upgrade can call:
// USDC.transferFrom(user, attacker, USDC.balanceOf(user))
```

C-02: adminTransferFrom Allows Owner to Transfer Tokens from Any User Wallet

C-02	`adminTransferFrom` Allows Owner to Transfer Tokens from Any User Wallet
Severity	Critical
Status	Fixed
Location	`contracts/RouterV2.sol: L11-18`

Description

The `adminTransferFrom` function allows the admin to call `safeTransferFrom(from, to, amount)`, moving all funds of any user who ever approved the Router, to any address. This function

is essentially a rug pull vector.

Users must give the Router approval for swap execution, but the approval uses the unlimited permit (see C-01), which is dangerous for users. If a user has performed at least one permit-swap, their funds can be drained.

Suggested Fix

Remove the `adminTransferFrom` function completely. There is already a function that recovers stuck funds — `rescueFunds()`, which can only withdraw funds that belong to the contract.

Proof of Concept

```
// 1. Victim executes permit-swap (or regular approve for Router)
victim.executeSwapWithPermit(
    target, data, USDC, WETH, 1000e6, deadline, v, r, s
);

// Now Router has unlimited allowance for all victim's USDC (type(uint256
   ).max)

// 2. Admin steals all victim's USDC
routerV2.adminTransferFrom(
    address(USDC),          // token
    address(victim),        // from - victim
    address(attacker),      // to - attacker's wallet
    USDC.balanceOf(victim) // amount - entire balance
);
```

C-03: Unrestricted delegatecall to Configurable swapTarget

C-03	Unrestricted `delegatecall` to Configurable `swapTarget`
Severity	Critical
Status	Acknowledged
Location	`updated/contracts/Router.sol: L298-304`

Description

Introduced in the updated contract. When `swapTarget` is set to a non-zero address, the Router executes a `delegatecall` to that address:

```
} else {
    (bool successRedirect, bytes memory redirectReturnData) =
        expectedTarget.delegatecall(
            abi.encodeWithSelector(
                _REDIRECT_SELECTOR, _tokenIn,
                _exec.approvalAmount, _data
            )
        );
}
```

A `delegatecall` executes the target's code in the context of the Router's storage. If `swapTarget` is set to a malicious contract (via compromised owner key or governance error), that contract can:

- Overwrite any storage variable of the Router (including `owner`).
- Drain all ETH and ERC-20 tokens held by the contract.
- `selfdestruct` the proxy implementation.

Although `setSwapTarget` is restricted to `onlyOwner`, the absence of a timelock (see M-01) means a single compromised key can immediately redirect all swaps through a malicious contract with full storage access.

Suggested Fix

Replace `delegatecall` with a regular `call`. If delegation is truly required, enforce a timelock and multisig governance on `setSwapTarget`, and restrict the callable selectors.

Proof of Concept

```
// 1. Attacker compromises owner key
// 2. Deploys MaliciousTarget with redirectToDiamond()
//    that calls: selfdestruct(attacker) or overwrites owner

// 3. Sets swapTarget
router.setSwapTarget(address(maliciousTarget));

// 4. Any user swap now executes attacker's code
//    in Router's storage context via delegatecall
user.executeSwap(target, data, USDC, WETH, 1000e6, 0);
// -> Router storage is compromised
```

2.2 High

No high severity vulnerabilities found.

2.3 Medium

M-01: Centralized Upgrade Authority

M-01	Centralized Upgrade Authority
Severity	**Medium**
Status	**Acknowledged**
Location	`contracts/proxy/FlakeRouter.sol: L75-77`

Description

The proxy admin can call `upgradeTo` / `upgradeToAndCall` immediately, with no timelock or staged governance. Compromise or abuse of the admin key means full protocol takeover.

Suggested Fix

- Move admin to a multisig wallet.
- Add timelock + public upgrade queue.
- Add emergency pause and rollback procedure.
- Enforce governance process for upgrades (not a single EOA key).

Proof of Concept

1. Attacker obtains the proxy admin key.
2. Calls `upgradeTo(maliciousImplementation)`.
3. Malicious implementation exposes fund-drain logic, called through the proxy.

M-02: No Slippage Protection

M-02	No Slippage Protection
Severity	**Medium**
Status	**Fixed**
Location	`contracts/Router.sol: L49-121`

Description

The swap parameters are subject to sandwich attacks by MEV bots. None of the swap functions accept a `minAmountOut` parameter to control the minimum amount of output tokens. The contract

never checks that the output tokens surpass the minimum.

Even though the calldata may contain slippage checks from the aggregator's level, at the Router level there are no guarantees.

Suggested Fix

Add a `uint256 _amountOutMin` parameter to all swap functions and enforce it after swap execution:

```
if (_tokenOut != address(0)) {
    uint256 received = tokenOutBalanceAfter - _exec.tokenOutBalanceBefore
    ;
    require(received >= _amountOutMin, "Insufficient output amount");
}
```

Proof of Concept

```
// Attacking MEV bot sees a tx in mempool for swap 10,000 USDC -> WETH

// 1. Frontrun: the bot buys WETH, moving the price up

// 2. The victim's transaction is executed at the inflated price
//    (victim receives 2.5 WETH instead of 3 WETH)

// 3. Backrun: the bot sells WETH at the inflated price
// Router doesn't check minAmountOut - transaction executes
// successfully with fund losses for the user
```

M-03: Obfuscated Target Address Through XOR

M-03	Obfuscated Target Address Through XOR
Severity	**Medium**
Status	**Acknowledged**
Location	`contracts/Router.sol:  L19-21, L41-43`

Description

The target address where the Router sends swaps (LI.FI Diamond:

`0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE`) is intentionally obfuscated through XOR of two constants:

```
uint160 private constant _TARGET_KEY_A =
    uint160(bytes20(hex"9b7c0f34d4a91f53a1d2c8f9b3e45d6a70c12e11"));
uint160 private constant _TARGET_KEY_B =
    uint160(bytes20(hex"894dd18221dd81a56fbb8b5bc6458e8d38ae60bf"));

function _targetAddress() internal pure returns (address) {
    return address(_TARGET_KEY_B ^ _TARGET_KEY_A);  // = 0x1231DEB6...
}
```

There is no legitimate reason to hide this target address. It undermines users' trust and makes external verification unnecessarily difficult.

Suggested Fix

Replace the obfuscation with the actual constant:

```
address private constant _LIFI_DIAMOND =
    0x1231DEB6f5749EF6cE6943a275A1D3E7486F4EaE;

function _targetAddress() internal pure returns (address) {
    return _LIFI_DIAMOND;
}
```

CONCLUSION

The initial audit of the Flake Router smart contracts revealed **5 findings**: 2 Critical and 3 Medium severity issues.

During the re-audit (March 16, 2026) the updated contracts were reviewed. **2 of 5** original findings have been fixed, **3 remain open**, and **1 new Critical** vulnerability was introduced:

- **C-02** (`adminTransferFrom`) — **Fixed**. The `RouterV2` contract with the dangerous function has been removed entirely.
- **M-02** (No Slippage Protection) — **Fixed**. A `_amountOutMin` parameter and corresponding `require` checks have been added to all swap functions.
- **C-01** (Unlimited Allowance) — **Acknowledged**. The fix attempt includes a fallback to `type(uint256).max`, which completely negates the intended remediation.
- **C-03** (delegatecall to swapTarget) — **Acknowledged**. New finding. The updated contract introduces a `delegatecall` to a configurable `swapTarget` address, granting full storage access to an external contract.
- **M-01** (Centralized Upgrade Authority) — **Acknowledged**. No timelock or multisig has been added.
- **M-03** (XOR Obfuscation) — **Acknowledged**. The XOR-based target address obfuscation remains as a legacy fallback.

Overall, the protocol still poses **significant risk to user funds**. The introduction of unrestricted `delegatecall` (C-03) combined with the lack of a timelock (M-01) creates a new attack surface that is potentially more dangerous than the original `adminTransferFrom`. The remaining Critical and Medium issues must be resolved before deployment or through an emergency upgrade. We strongly recommend removing the `delegatecall` path, eliminating the `type(uint256).max` fallback in permit, adopting a multisig governance model with timelock, and conducting a follow-up audit after remediation.

ABOUT BUGBLOW

BugBlow is a security firm specializing in comprehensive audits and penetration testing for Web3 and traditional applications. Founded in 2024, BugBlow helps projects identify and eliminate vulnerabilities before they reach production — whether the code runs on-chain or off-chain.

Our team comprises security engineers and researchers with deep expertise in smart contract internals, backend architecture, cryptographic protocols, and application security. We combine manual review with automated tooling and AI-assisted analysis to deliver thorough, high-signal audit reports.

Why BugBlow

- **Full-Stack Coverage:** We audit both on-chain smart contracts (Solidity, Solana, FunC) and off-chain application code (APIs, backends). Most vulnerabilities live at the boundary between the two — we cover both sides.
- **Penetration Testing:** Beyond code review, we simulate real-world attack scenarios against live and staging environments to uncover issues that static analysis alone cannot find.
- **Adversarial Mindset:** Every audit includes Proof-of-Concept exploits for critical findings. We don't just flag theoretical risks — we demonstrate impact.
- **Developer-Friendly Process:** We stay in direct communication throughout the engagement, share preliminary findings early, and work with your team to verify fixes before the final report.

Our Services

- **Smart Contract Audits:** Security assessment of on-chain code across EVM, TON, and Solana ecosystems. Includes manual review, automated analysis, and formal verification where applicable.
- **Off-Chain Audits:** Security review of server-side applications, APIs, microservices, and infrastructure code. Covers authentication, authorization, race conditions, input validation, data integrity, and business logic flaws.
- **Penetration Testing:** Black-box and gray-box testing of web applications, APIs, and infrastructure. Simulates external and internal threat actors to identify exploitable attack paths.
- **Security Consulting:** Architecture review, threat modeling, and security best practices guidance for teams building in Web3 and beyond.

Contact Information

Web `https://bugblow.com`

GitHub `https://github.com/BugBlow/audits`

X `https://x.com/BugBlow`